

DATABASE SYSTEMS

Chap 4~ dbms_manipulation: SQL LANGUAGE

5.1. SQL LANGUAGE

- SQL is a programming language for Relational Databases.
- It is designed over relational algebra
- SQL comes as a package with all major distributions of RDBMS.
- SQL comprises both **data definition** and **data manipulation languages**.

History of SQL Standard

- SQL stands for "Structured Query Language"
- The structured Query Language is a relational database language(SQL is a language used to interact with the database).
- First Version was developed at IBM by Donald D. Chamberlin and Raymond F. Boyce. [SQL]
- The first commercial DBMS that supported SQL was Oracle in 1979
- SQL is a nonprocedural language

History of SQL Standard

- SQL was originated from the research project conducted by IBM in San Jose, California in the early 1970s.
- The aim was to develop an experimental RDBMS which would lead to a marketable product.
- A language SEQUEL (Structured English QUERy Language) was chosen.
- In 1980 minor revisions were made to SEQUEL, and it was renamed SQL.
- In 1983, IBM released a second SQL-based RDBMS called DB2, which ran under the MVS OS.
- Later on, SQL language was standardized by ANSI and ISO.

Data-types in SQL

- In relational model the data are stored in the form of tables.
- A table is composed of rows and columns.
- When we create a table we must specify a data-type for each of its columns.
- The data-types define the domain of values that each column can take.

Data-types in SQL

✓ String

- ✓ We have CHAR and VARCHAR data-types.
- ✓ Character data type store data which are words, and free-form text, in the database character set

✓ CHAR

- Specifies a **fixed-length character string**.
- The syntax is CHAR (n)~ fixed length character data

✓ VARCHAR

- Specifies a **variable-length character string**
- The syntax of VARCHAR2 (n)~ Variable length character data

Data-types in SQL

✓ Number

- Stores zero, positive, and negative fixed and floating point numbers
- Syntax to store fixed-point number is NUMBER (p,q)
- Syntax to specify an integer is NUMBER(p)

✓ DATE

- Used to store the date and time information
- The ANSI date literal contains no time portion, and must be specified in YYYY-MM-DD format where Y stands for Year, M for month, and D for data

Data-types in SQL

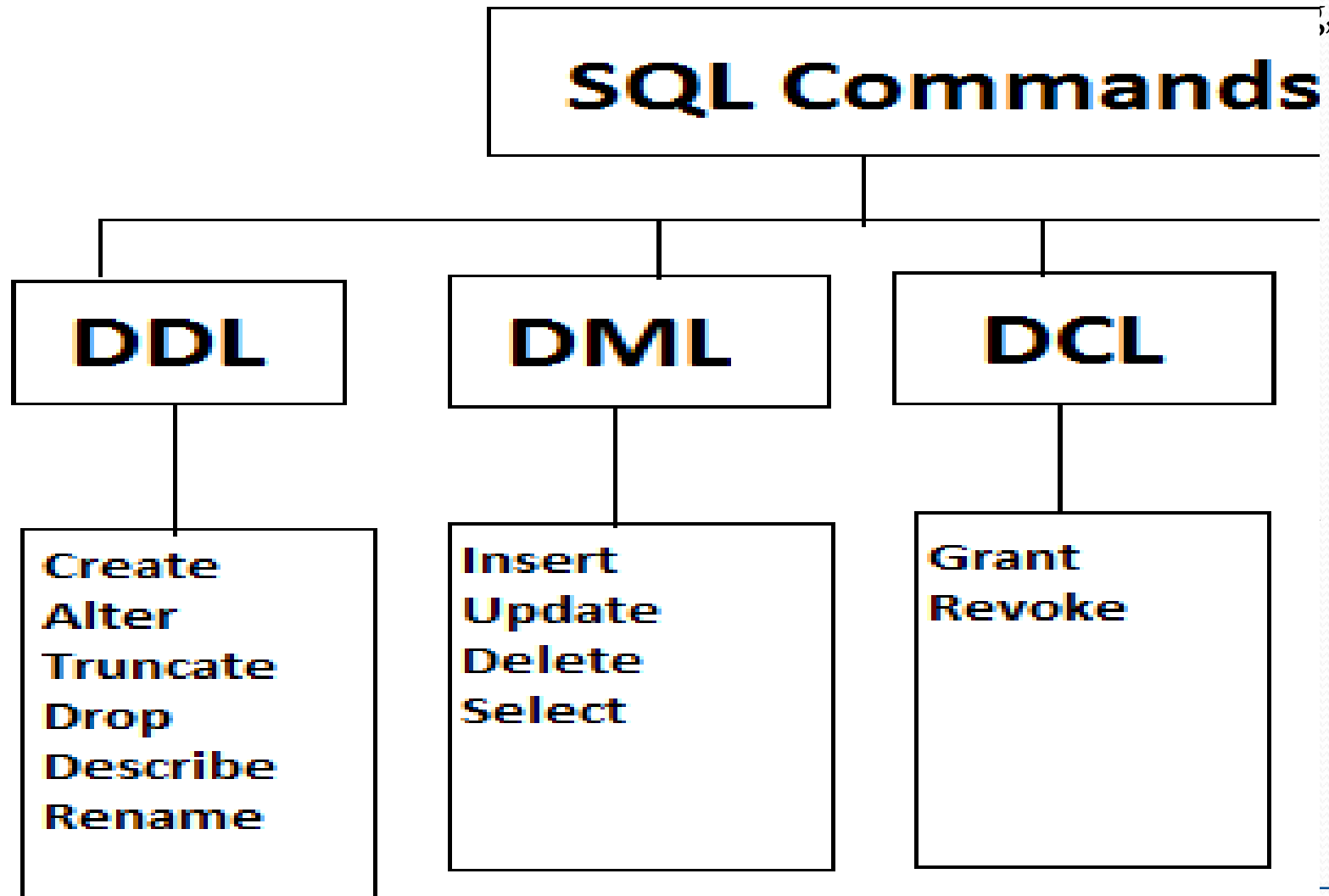
✓ TIME STAMP

- Used to store both date and time
- It stores the year, month, and day of the DATE datatype, and also hour, minute and second values

✓ LOB Datatype

- Multimedia data like sound, picture, and video need more storage space.
- BLOB, CLOB and BFILE allows us to store large block of data
- BLOB stores unstructured **binary data** in database up to 4G
- CLOB stores up to 4G of **character data**
- BFILE stores unstructured binary data in OS files outside the database, it can store up to 4G of data

Commands in SQL



5.2. Commands in SQL

- SQL commands can be classified in to three types:
 - ✓ Data Definition Language commands (DDL)
 - Commands are used to define a database, including creating, altering, and dropping tables and establishing constraints
 - ✓ Data Manipulation Language commands (DML)
 - Commands are used to maintain and query a database, including updating, inserting, modifying and querying data
 - ✓ Data Control Language commands (DCL)
 - Commands are used to control a database including administering privileges and saving of data.
 - Are used to determine whether a user is allowed to carry out a particular operation or not

Get it started

1. Install xampp:

XAMPP stands for:

- **X:** Cross-Platform (works on Windows, Linux, and MacOS)
- **A:** Apache (an open-source web server)
- **M:** MySQL (database management system, though newer versions use MariaDB)
- **P:** PHP (server-side scripting language)
- **P:** Perl (another programming language)

Get it started

2. Run the servers:

XAMPP Control Panel v3.2.3 [Compiled: Mar 7th 2019]

XAMPP Control Panel v3.2.3

Service	Module	PID(s)	Port(s)	Actions
☐	Apache	16180 5764	80, 443	Stop Admin Config Logs
☐	MySQL	15496	3306	Stop Admin Config Logs
☐	FileZilla			Start Admin Config Logs
☐	Mercury			Start Admin Config Logs
☐	Tomcat			Start Admin Config Logs

Config
Netstat
Shell
Explorer
Services
Help
Quit

8:26:59 PM [main] All prerequisites found
8:26:59 PM [main] Initializing Modules
8:26:59 PM [main] Starting Check-Timer
8:26:59 PM [main] Control Panel Ready
8:28:22 PM [mysql] Attempting to start MySQL app...
8:28:23 PM [mysql] Status change detected: running
8:33:29 PM [Apache] Attempting to start Apache app...
8:33:29 PM [Apache] Status change detected: running

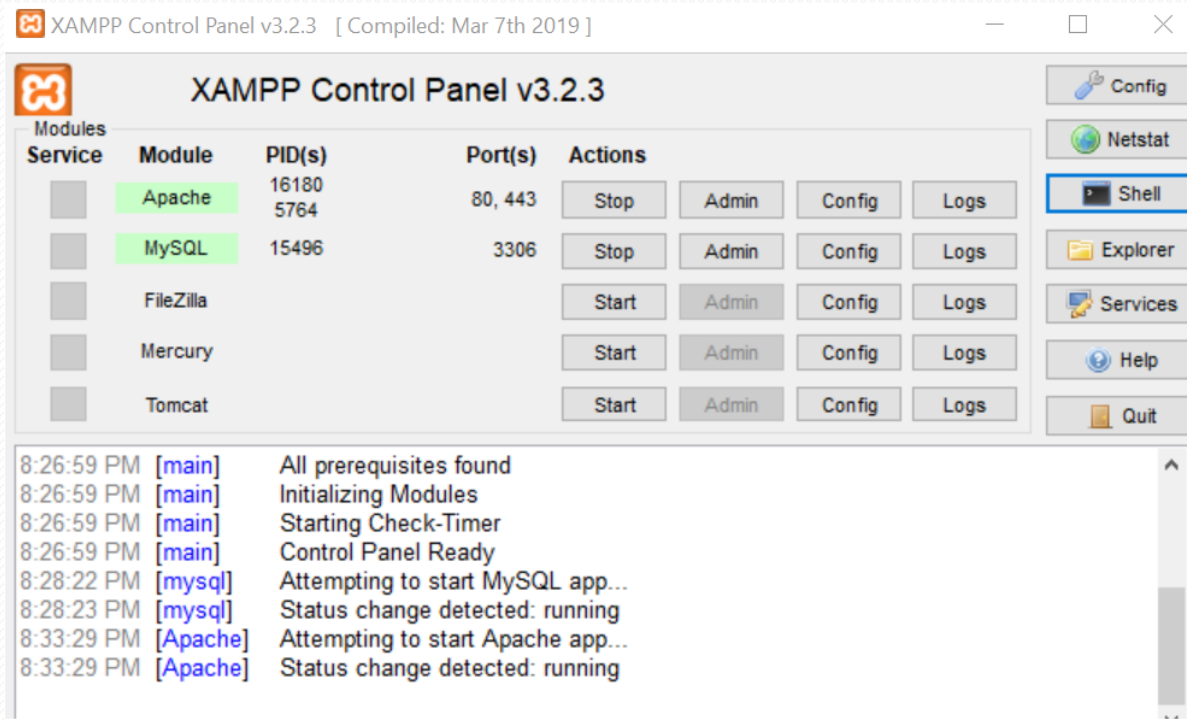
Get it started

3. Open Command line mode:

START button -TYPE cmd then move to

`cd C:\xampp\mysql\bin`

Or



Get it started

4. Access to the database server: `mysql -u root -p`

DATA DEFINITION LANGUAGE(DDL)

- Data Definition Language (DDL) properties of SQL, are used to design and modify database schema. SQL uses the following set of commands to define database schema .
- CREATE
- DROP
- ALTER

CREATE

Creates new databases, tables and views from RDBMS.

For example;

1. Create database marks;
2. Create table article;
3. Create view for_students;

1. Create A NEW database command

To begin, you must first CREATE a database using the following SQL statement:

```
CREATE DATABASE database_name
```

Depending on the version of SQL being used the following statement is needed to begin using the database:

```
USE database_name
```

2. Create Table Command

- ✓ The CREATE TABLE command is used to implement the schemas of individual relations.
- ✓ Steps in creating table
 1. Identify data types for attributes
 2. Identify columns that can and cannot be null
 3. Identify columns that must be unique
 4. Identify primary key-foreign key mates
 5. Determine default values
 6. Identify constraints on columns (domain specifications)
- ✓ Syntax to create the table

```
CREATE TABLE table_name(column name1    data type    (size),
                           column name2    data type    (size)
                           column nameN    data type    (size));
```

Example of a table

```
CREATE TABLE authors  
  (auth_id int(9) not null,  
   auth_name char(40) not null)
```

auth_id	auth_name
(9 digit int)	(40 char string)

Table & database Description

- To see all the tables in a database
 - show databases;
 - show tables ;
- To see the description each table
 - desc table-name;

DROP

Drops commands, views, tables, and databases from RDBMS.

For example;

1. Drop object_type object_name;
2. Drop database marks;
3. Drop table article;
4. Drop view for_students;

TRUNCATE vs DROP TABLE

- ✓ The Truncate Table command removes all the rows from the table.
- ✓ The truncate table also releases the storage space used by the table.

TRUNCATE TABLE table-name

- ✓ **Dropping a Table**

- The definition of the table as well as the contents of the table is deleted by issuing DROP TABLE command.

DROP TABLE table-name ;

The DROP statement is also used to delete an entire database.

Table Modification Commands

- ✓ The ALTER TABLE statement is used to add, delete, or modify columns in an existing table.

- ✓ Adding a column to the Table

ALTER table table_name

ADD column column_name data type (size);

- Modifying the column of the Table (To change the data type of a column in a table,)

ALTER table table_name

MODIFY column column_name data type;

- Deleting the column of the Table

ALTER table table_name

DROP COLUMN column-name;

Imposition of Constraints

- ✓ Constraints are basically used to impose rules on the table, whenever a row is inserted, updated, or deleted from the table.
- ✓ Constraints prevent the deletion of a table if there are dependencies.
- **NOT NULL Constraint**
 - Whenever an attribute is declared as NOT NULL, then it specifies that the attribute cannot contain a NULL value

CREATE TABLE table name

*(column name1 data type (size) NOT NULL,
column name2 data type (size),
column nameN data type (size));*

Imposition of Constraints

- **UNIQUE Constraint**

- The UNIQUE constraint imposes that every value in a column or set of columns be unique
- It means that no two rows of a table can have duplicate values in a specified column or set of columns.

CREATE TABLE table name

*(column name1 data type (size) ,
column name2 data type (size) UNIQUE,
column nameN data type (size));*

Imposition of Constraints

- **PRIMARY KEY constraint**
 - It is used to identify each row of the table uniquely.

CREATE TABLE table name

*(column name1 data type (size) ,
column name2 data type (size) ,
column nameN data type (size))
primary key(column name1);*

- **On existing table**

ALTER TABLE TABLE-NAME ADD PRIMARY KEY (Column name1);

Imposition of Constraints

- *FOREIGN KEY constraint*
- Specifies that the value of an attribute in one table depends on the value of the same attributes in another table

CREATE TABLE table name1

(column name1 data type (size),

column name2 data type (size),

column name21 data type (size),

column nameN data type (size))

primary key(column name1);

FOREIGN KEY (column name21)REFERENCES table name2(column name21)

✓ On existing table

ALTER TABLE table-name1
ADD FOREIGN KEY (column name21) REFERENCES table name2
(column name21);

Imposition of Constraints

- Difference between UNIQUE and NOTNULL constraint

NOTNULL constraint	UNIQUE constraint
It will not accept NULL values	Can accept NULL values
It will accept duplicate values	Will not accept duplicate values

- Difference between UNIQUE and PRIMARY KEY constraint

PRIMARY KEY constraint	UNIQUE constraint
It will not accept NULL values	Will accept NULL values
Only one PRIMARY KEY can be defined for each table	More than one UNIQUE constraint can be defined for each table

DATA MANIPULATION LANGUAGE

- Data Manipulation Language (DML) properties allows SQL to store and retrieve data from database. SQL contains the following set of commands in its DML section:
 - INSERT INTO/VALUES
 - UPDATE/SET/WHERE
 - DELETE FROM/WHERE
 - SELECT/FROM/WHERE

CONT...

- These basic constructs allow database programmers and users to enter data and information into the database and retrieve efficiently using a number of filter options.

Insert data in a table command

SYNTAX: INSERT INTO/VALUES:

command is used for inserting values into the rows of a table (relation).

Syntax:

- `INSERT INTO table table_name (column1 , column2, column3 ... ) VALUES (value1 , value2, value3 ... )`

OR

- `INSERT INTO table table_name VALUES (value1, value2, ... )`

EXAMPLE

1. INSERT INTO Books(Author, Subject) VALUES ("anonymous", "computers");

OR

2. INSERT INTO Books VALUES ("anonymous", "computers");

Example: Employee table

EmpID	EmpName	Salary	Age	DeptID
101	John	50000	28	1
102	Mary	40000	32	2
103	Peter	60000	30	1
104	Jane	55000	27	3
105	Paul	45000	25	5
106	Alice	42000	29	2
107	Bob	70000	35	6
108	Clare	65000	31	4
109	David	48000	26	7
110	Eva	39000	24	8

Department table

DeptID	DeptName	Location
1	IT	Kigali
2	HR	Huye
3	Finance	Musanze
4	Marketing	Rubavu
5	Sales	Kigali
6	Engineering	Muhanga
7	Support	Nyagatare

Update the table data

Syntax: UPDATE/SET/WHERE

This command is used for updating or modifying the values of columns in a table (relation).

Syntax: Updating the data in the table

```
UPDATE table_name  
SET attribute_value = new_value  
WHERE condition;
```

EXAMPLE

```
UPDATE Student  
SET Age = 21  
WHERE StudentID = 1;
```

Update the table data

Updating Multiple Columns:

Syntax: Updating the data in the table

```
UPDATE table_name  
SET attribute_value = new_value,  
attribute_value=new_value  
WHERE condition;
```

EXAMPLE

```
UPDATE Student  
SET Name = 'Ange',  
Age = 23  
WHERE StudentID = 2;
```

Delete table contents

- **DELETE/FROM/WHERE:** This command is used for removing one or more rows from a table (relation).

Syntax: DELETE FROM table_name [WHERE condition];

EXAMPLE

1. DELETE FROM Books WHERE Author="David";
2. DELETE FROM Student WHERE Age = 22 AND Class = 'A';
3. DELETE FROM Employee WHERE Department = 1 AND Salary < 300000;
1. DELETE FROM Employee WHERE Department = 2 AND Salary < 300000 AND Age < 30;

Displaying Table contents

Syntax: **SELECT/FROM/WHERE**

- **SELECT:** It is similar to the projection operation of relational algebra. It selects the attributes based on the condition described by WHERE clause.
- **FROM** – This clause takes a relation name as an argument from which attributes are to be selected/projected. In case more than one relation names are given, this clause corresponds to Cartesian product.
- **WHERE** – This clause defines predicate or conditions, which must match in order to qualify the attributes to be projected.

Displaying Table contents

- **Select** author_name **From** book_author **Where** age > 50;
- This command will yield the names of authors from the relation **book_author** whose age is greater than 50.

Displaying Table contents

- The **SELECT** statement with multiple conditions is used to retrieve data from a table using **WHERE** with **AND**, **OR**, **IN** and **order by** .
- Examples :
 1. **SELECT *FROM** Employee **WHERE** Department = 1 **AND** Salary > 200000;
 2. **SELECT *FROM** Employee **WHERE** Department = 5 **OR** Salary > 250000;

Displaying Table contents

3. **SELECT *FROM** Employee **WHERE** Salary **BETWEEN** 200000 **AND** 300000;
4. **SELECT *FROM** Employee **WHERE** DeptID **IN** (1, 2, 5);
5. **SELECT *FROM** Employee **WHERE** (Department = 3 **OR** Department = 5)**AND** Salary > 200000 **ORDER BY** Age;

Aggregate Functions

- ✓ SQL provides four built-in functions to facilitate query processing.

Serial	Built-in function	Use
1	<i>GROUP BY</i>	Used to group rows to compute group-statistics.
2	<i>HAVING command</i>	Used to select the group, it restricts the groups according to a specified condition
3	<i>SORTING of Results</i>	ORDER BY is used to sort the result in ascending or descending order
4	<i>Using BETWEEN</i>	It is used to perform range queries

- **SELECT** EmpID, Name, Salary **FROM** Employee **WHERE** Salary **BETWEEN** 200000 **AND** 400000 **ORDER BY** Salary **DESC**;

Displaying Table contents

With **AGGREGATE OPERATORS**

- In addition to simply retrieving data, we often want to perform some computation or summarization.
- SQL allows the use of arithmetic expressions.
- We now consider a powerful class of constructs for computing aggregate values which are following:

CONT...

Serial	Built-in function	Use
1	<i>COUNT</i>	To count the number of rows of the relation
2	<i>MAX</i>	To find the maximum value of the attribute (column)
3	<i>MIN</i>	To find the minimum value of the attribute
4	<i>SUM</i>	To find the sum of values of the attribute provided the data type of the attribute is number
5	<i>AVG</i>	To find the average of n values, ignoring null values

Aggregate Functions

✓ Syntax for aggregate functions

```
SELECT COUNT (*)  
FROM table_name;
```

```
SELECT COUNT  
(DISTINCT  
attribute_name)  
FROM table_name;
```

```
SELECT aggregate  
function( attribute name,)  
FROM table name GROUP  
BY attribute name;
```

```
SELECT MAX ( attribute  
name)  
FROM table name;
```

```
SELECT AVG ( attribute  
name)  
FROM table name;
```

```
SELECT MIN ( attribute  
name)  
FROM table name;
```

EXAMPLES

1. SELECT SUM(Salary) AS TotalSalary FROM Employee;
2. 4. SELECT MAX(Age) AS OldestAge FROM Employee;
3. SELECT MIN(Salary) AS LowestSalary FROM Employee;
4. SELECT SUM(Salary) AS IT_TotalSalary FROM Employee
WHERE DeptID = 5;
5. SELECT AVG(Age) AS IT_AverageAge FROM Employee
WHERE DeptID = 7;
5. SELECT DeptID, SUM(Salary) AS TotalSalary FROM Employee
GROUP BY DeptID;

Aggregate Functions

- ✓ SQL provides four built-in functions to facilitate query processing.

Serial	Built-in function	Use
1	<i>GROUP BY</i>	Used to group rows to compute group-statistics.
2	<i>HAVING command</i>	Used to select the group, it restricts the groups according to a specified condition
3	<i>SORTING of Results</i>	ORDER BY is used to sort the result in ascending or descending order
4	<i>Using BETWEEN</i>	It is used to perform range queries

Aggregate Functions

✓ Syntax for aggregate functions

```
SELECT ( attribute name,  
        aggregate function)  
FROM table name  
GROUP BY attribute  
        name  
HAVING condition;
```

```
SELECT *  
FROM table name  
ORDER BY  
attribute name ASC;
```

```
SELECT *  
FROM table name  
WHERE attribute name  
BETWEEN value1 AND  
        value2;
```

```
SELECT *  
FROM table name  
ORDER BY  
attribute name DESC;
```

GROUP BY Clause

- The SQL **GROUP BY** is used in collaboration with the **SELECT** statement to arrange identical data into groups.
- The **GROUP BY** clause follows the **WHERE** clause in a **SELECT** statement and precedes the **ORDER BY** clause.

SYNTAX: select **col1, col2, col3** from **table_name** **where**
[condition] group by col1;

E.g: **SELECT** DeptID, **COUNT(*)** **AS** TotalEmployees **FROM**
Employee **GROUP BY** DeptID;

ORDER BY CLAUSE

- The **SQL ORDER BY** is used in collaboration with the **SELECT** statement to sort results either in ascending or descending order.
- The **ORDER BY** clause follows the **WHERE** clause in a **SELECT** statement and comes after the **GROUP BY** clause.
- **SYNTAX:** **select col1, col2, col3 from table_name where [condition] group by col1, col2 order by col1 asc|desc.**

E.g: **SELECT** EmpName, Salary **FROM** Employee **ORDER BY** Salary **DESC**;

HAVING CLAUSE

- The HAVING clause enables to specify conditions that filter which group results appear in the final results.
- Syntax: **SELECT FROM** table_name **WHERE** [condition] **GROUP BY HAVING ORDER BY**.
- Eg: **SELECT DeptID, COUNT(*) AS TotalEmployees FROM Employee GROUP BY DeptID HAVING COUNT(*) > 3;**

Between clause

- In between is used to filter data in a range
- E.g. `SELECT EmpName, Salary FROM Employee WHERE Salary BETWEEN 45000 AND 60000;`

DBMS JOINS

- **Join** is a combination of a Cartesian product followed by a selection process.
- A **join** is used to fetch data from two or more tables, which are joined to appear as a single set of data.
- The following are the types of JOIN that we can use in SQL.
 - ❖ Inner
 - ❖ Left
 - ❖ Right
 - ❖ Full

INNER JOIN

- An inner join includes only those tuples with matching attributes (from different tables) and the rest are discarded in the resulting relation.
- The inner join creates a new result table by combining column values of two tables based upon the join-predicate.

SYNTAX: SELECT table1.col1, table2.col2, ... FROM table1 INNER JOIN table2 ON = table2.field. table1.common_field

Eg: Show only employees with their departments

- **SELECT** e.EmpName, e.Salary, d.DeptName, d.Location **FROM** Employee **INNER JOIN** Department d **ON** e.DeptID = d.DeptID;
- **SELECT *FROM** Employee **NATURAL JOIN** Department;
- **SELECT** e.EmpName, e.Salary, d.DeptName**FROM** Employee e**JOIN** Department d**ON** e.DeptID = d.DeptID**WHERE** e.Salary > 45000;

LEFT JOIN

- It returns all rows from the left table, even if there are no matches in the right table.
- This means that a left join returns all values from the left table, plus matched values from right table or null in case of no matching join predicate.

Syntax: `select table1.col1, table2.col2 from table1 left join table2 on table1.column_field = table2.column_field.`

Eg: `SELECT e.EmpName, d.DeptName FROM Employee e
LEFT JOIN Department d ON e.DeptID = d.DeptID;`

- Show all employees, even if department is missing

RIGHT JOIN

- It returns all rows from the right table, even if there are no matches left table.
- This means a right join returns all the values from the right table, plus matched values from the left table or NULL in case of no matching join predicate.

Syntax: `select table1.col1, table1.col2, table2.col1, ... from table1
right join table2 on table1.common_field =
table2.common_field;`

Eg: `SELECT e.EmpName, d.DeptName FROM Employee e
RIGHT JOIN Department d ON e.DeptID = d.DeptID;`

- Show all departments, even if no employees exist

FULL JOIN

- It combines the results of both left and right outer joins. The joined table will contain all records from both tables and fill in NULL for missing matches on either side.
- **Syntax: `select table1.col1, table2.col1,... from table1 full join table2 on table1.common_field = table2.common_field;`**
- **not supported by mysql**

EXAMPLE

- Consider the table for customers and orders considered in the previous examples;
- **Syntax: Select id, name, amount, date from customers full join orders on customers.id= order.customer_id;**
- This query will result in combined results of both tables and null value will be given instead.

CONT...

- If your database does not support FULL JOIN like MySQL database, then you can use UNION ALL clause as follows:

**Select id, name, amount, date from customers left join
orders on customers.id= orders_customer_id**

UNION ALL

**Select id, name, amount, date from customers right join
orders on customers.id = orders.customers_id;**

Joining three tables

- Referring to the following tables (employee, department, project)
- Create a query to Display employees, their departments, and projects name for departments located in Kigali.

```
SELECT Employee.EmpName,  
       Department.DeptName,  
       Department.Location,  
       Project.ProjectName  
FROM Employee  
INNER JOIN Department  
ON Employee.DeptID = Department.DeptID  
INNER JOIN Project  
ON Employee.EmpID = Project.EmpID  
WHERE Department.Location = 'Kigali';
```

Joining multiple tables

- The number of tables you can join depends on the database system you are using

Examples:

- **Mysql support** Up to 61 tables in a join
 - **Microsoft SQL Server** Up to 256 tables
- Although many tables can be joined, usually developers join: 2 tables , 3 tables , 4 tables and sometimes 5-10 tables in complex systems.
- Too many joins can Slow down queries

VIEW

- View is a pseudo-table or virtual table .
- It don't store data but it just display the data
- The data are derived from one or more base tables
- It is to be noted that all views are not updatable

CREATE VIEW view-name
AS SELECT attribute list
FROM table (s),
WHERE condition (s);

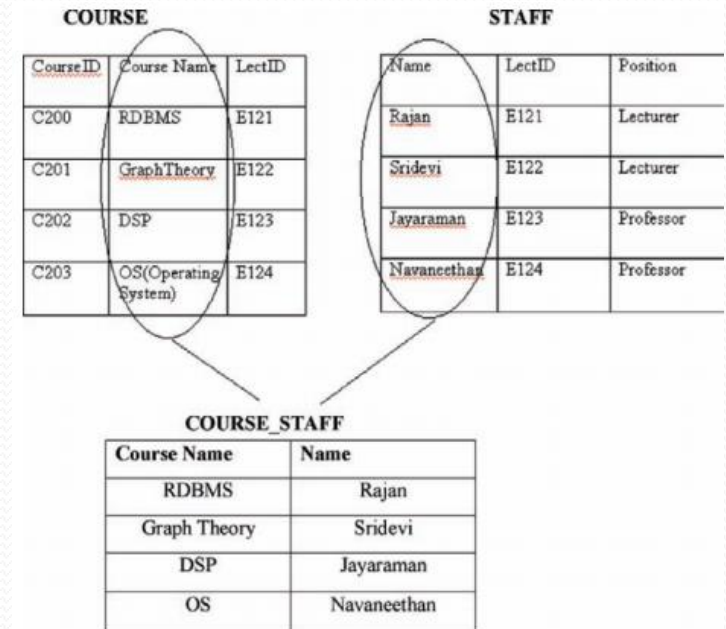
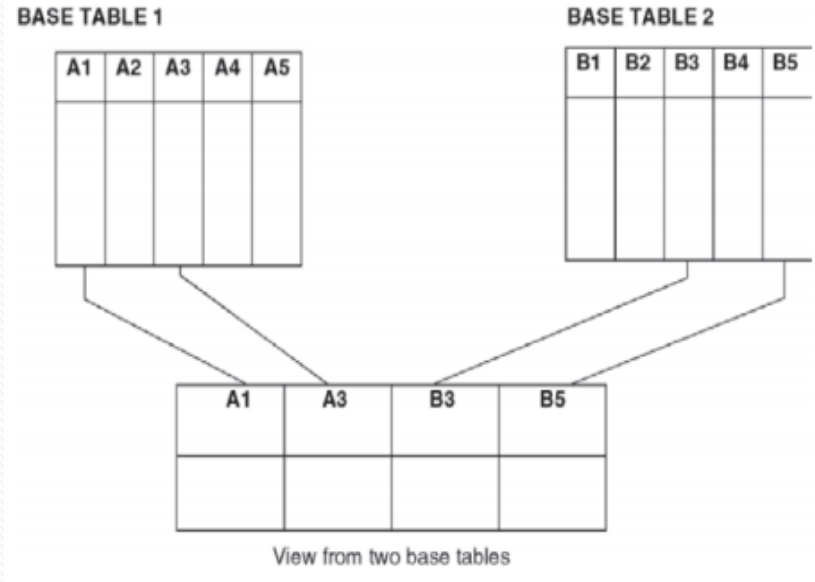
RECORD				
S.I. No	Name	Age	GPA	Institution
1	Anbalagan	22	9.2	PSG
2	Balu	22	9.4	PSG
3	Dinesh	22	8.4	CIT
4	Karthik	21	8.5	REC
5	Kumar	22	8.7	MIT
6	Kishore	22	8.8	MIT
7	Rajan	22	9.1	PSG
8	Lavanya	21	9.1	CIT

GPA >9.0

PLACED		
Name	Age	Institution
Anbalagan	22	PSG
Balu	22	PSG
Rajan	22	PSG
Lavanya	21	CIT

VIEW

Views from Multiple Tables



CREATE VIEW view-name
AS SELECT attribute list
FROM table 1, table2
WHERE condition;

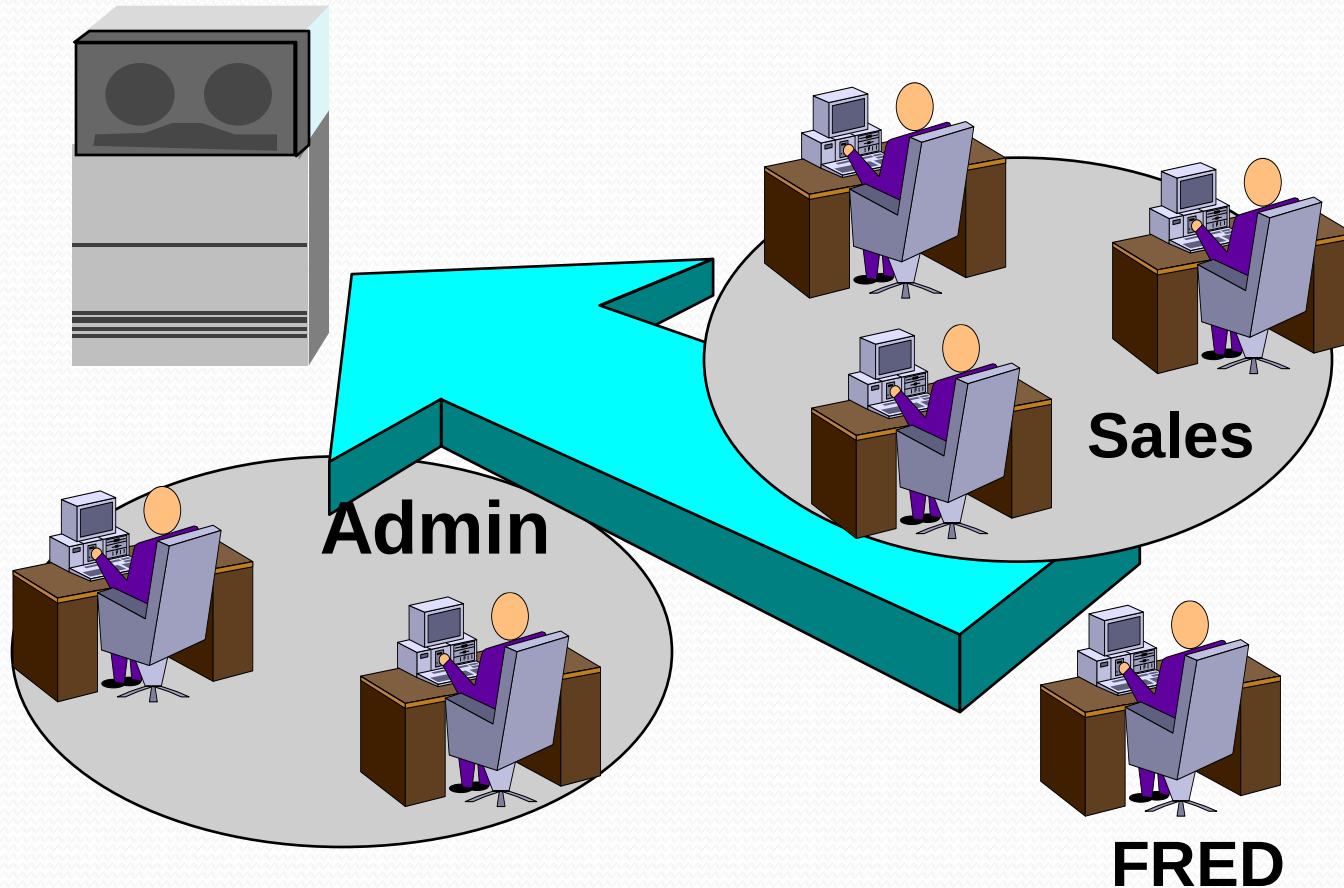
- **Advantage of VIEW**
- The main advantage of view are:
 1. Improved security
 2. Less complexity
 3. Convenience
 4. Customization

DIFFERENCES BETWEEN DDL & DML

DDL	DML
DDL commands are used in the beginning to create tables and whenever there is a need to change structure of a table or to destroy a table completely.	DML commands do not change the structure of a table but only to add, delete, update or search records from the table.
DDL commands are normally executed by database designer/administrators.	DML commands are normally executed by person who has the responsibility to manage database every day.
DDL commands are not frequently used.	DML commands are used frequently used to keep database updated according to changes in real life.

Data Control Language

- ✓ SQL commands for security mechanisms in an RDBMS
- ✓ Identify users



Data Control Language

- ✓ Allowable Privileges
 - SELECT, INSERT, UPDATE, DELETE
 - CREATE Table, View, Procedure,
- ✓ The owner/creator of a table automatically has all the privileges

Data Control Language

✓ GRANT / REVOKE

- GRANT privilege ON table-name TO list [WITH GRANT OPTION]
- For example
 - ❖ GRANT ALL ON dept TO John
 - ❖ GRANT SELECT ON dept TO sally WITH GRANT OPTION
 - ❖ GRANT SELECT, UPDATE, INSERT ON dept TO Jim, Mike
- ❖ REVOKE privilege ON table-name FROM list [CASCADE]
 - E.g. REVOKE SELECT ON dept FROM Sally CASCADE
 - However largely done these days via checkboxes in grids in GUI admin tools



- END